

LANGAGE de REQUETE MongoDB MQL et AGREGATION MongoDB

Introduction

Comparaison SQL (CREATE, ALTER, SELECT, DROP en SQL) - MongoDB

Pipeline d'agrégation MongoDB => Opérations d'agrégation de données courantes en SQL

INTRODUCTION

MQL : syntaxe simple pour interroger les documents au sein d'une même collection => collection unique

MQL ne permet pas de traiter les agrégations ou la gestion complexes de documents.

Il faut structurer toutes les opérations complexes en petites opérations via des opérateurs.

Chaque aspect correspond à une étape et tout se déroule en chaîne.

Les fonctions d'agrégations permettent de manipuler les données renvoyées par une requête MongoDB.

Les langages de requête MongoDB sont conçus pour les collections de requêtes uniques.

Le pipeline d'agrégation de MongoDB est construit sous forme d'étapes ou chaque étape opère sur les documents de l'étape précédente.

L'agrégation est utilisée lorsque le traitement de données complexes est requis.

COMPARAISON SQL (CREATE, ALTER, SELECT, DROP en SQL) – MongoDB

Terminologie/ Concept SQL Terminologie/ Concept MongoDB

SQL	MongoDB
Database	database
Table	collection
Row	document
Column	field
Index	index
Primary key	Primary key (_id field)

Exemples Instructions SQL // Instructions MongoDB

⇒ *On travaille sur une collection nommée people*

```
{  
  _id: ObjectId("509a8fb2f3f4948bd2f983a0"),  
  user_id: "abc123",  
  age: 55,  
  status: 'A'  
}
```

Création de table SQL <pre>CREATE TABLE people (Id NOT NULL AUTO_INCREMENT, user_id Varchar(30), age Number, status char(1), PRIMARY KEY (id))</pre>	Déclaration de Schéma MongoDB <pre>db.createCollection("people") db.people.insertOne({ user_id: "abc123", age: 55, status: "A" })</pre>
--	---

Creation INDEX en SQL	Creation index MongoDB
<pre>CREATE INDEX idx_user_id_asc ON people(user_id)</pre> <pre>CREATE INDEX idx_user_id_asc_age_desc ON people(user_id, age DESC)</pre>	<pre>db.people.createIndex({ user_id: 1 })</pre> <pre>db.people.createIndex({ user_id: 1, age: -1 })</pre>

Suppression en SQL	Suppression MongoDB
<pre>DROP TABLE people</pre>	<pre>db.people.drop()</pre>

Insérer en SQL	Insérer MongoDB
<pre>INSERT INTO people(user_id, age, status) VALUES ("bcd001", 45, "A") ;</pre>	<pre>db.people.insertOne({ user_id: "bcd001", age: 45, status: "A" })</pre> Remarque : <pre>db.people.insertMany([{ user_id: "bcd001", age: 45, status: "A" }, { user_id: "xxx", age: 46, status: "B" }, { _user_id: "yyy", age: 47, status: "A" }]);</pre>

Sélectionner en SQL

```
SELECT *  
FROM people
```

```
SELECT id,  
       user_id,  
       status  
FROM people
```

```
SELECT user_id, status  
FROM people
```

```
SELECT *  
FROM people  
WHERE status = "A"
```

```
SELECT user_id, status  
FROM people  
WHERE status = "A"
```

```
SELECT *  
FROM people  
WHERE status != "A"
```

```
SELECT *  
FROM people  
WHERE status = "A"  
AND age = 50
```

```
SELECT *  
FROM people  
WHERE status = "A"
```

Sélectionner MongoDB

```
db.people.find()
```

```
db.people.find(  
  {},  
  { user_id: 1, status: 1 }  
)
```

```
db.people.find(  
  {},  
  { user_id: 1, status: 1, _id: 0 }  
)
```

```
db.people.find(  
  { status: "A" }  
)
```

```
db.people.find(  
  { status: "A" },  
  { user_id: 1, status: 1, _id: 0 }  
)
```

```
db.people.find(  
  { status: { $ne: "A" } }  
)
```

```
db.people.find(  
  { status: "A",  
    age: 50 }  
)
```

```
db.people.find(  
  { $or: [ { status: "A" }, { age: 50 } ] }  
)
```

OR age = 50

```
SELECT *  
FROM people  
WHERE age > 25
```

```
SELECT *  
FROM people  
WHERE age < 25
```

```
SELECT *  
FROM people  
WHERE age > 25  
AND age <= 50
```

```
SELECT *  
FROM people  
WHERE user_id like "bc%"
```

```
SELECT *  
FROM people  
WHERE status = "A"  
ORDER BY user_id ASC
```

```
SELECT *  
FROM people  
WHERE status = "A"  
ORDER BY user_id DESC
```

```
SELECT COUNT(*)  
FROM people
```

)

```
db.people.find(  
  { age: { $gt: 25 } }  
)
```

```
db.people.find(  
  { age: { $lt: 25 } }  
)
```

```
db.people.find(  
  { age: { $gt: 25, $lte: 50 } }  
)
```

```
db.people.find( { user_id: /^bc/ } )
```

```
db.people.find( { status: "A" } ).sort(  
  { user_id: 1 } )
```

```
db.people.find( { status: "A" } ).sort( {  
  user_id: -1 } )
```

```
db.people.count()  
db.people.find().count()
```

<pre>SELECT COUNT(user_id) FROM people</pre>	<pre>db.people.count({ user_id: { \$exists: true } })</pre>
<pre>SELECT COUNT(*) FROM people WHERE age > 30</pre>	<pre>db.people.count({ age: { \$gt: 30 } })</pre>
<pre>SELECT DISTINCT(status) FROM people</pre>	<pre>db.people.aggregate([{ \$group : { _id : "\$status" } }]) db.people.distinct("status")</pre>
<pre>SELECT * FROM people LIMIT 1</pre>	<pre>db.people.findOne() db.people.find().limit(1)</pre>

Mise à jour en SQL	Mise à jour MongoDB
<pre>UPDATE people SET status = "C" WHERE age > 25</pre>	<pre>db.people.updateMany({ age: { \$gt: 25 } }, { \$set: { status: "C" } }</pre>
<pre>UPDATE people SET age = age + 3 WHERE status = "A"</pre>	<pre>db.people.updateMany({ status: "A" }, { \$inc: { age: 3 } })</pre>

Supprimer les enregistrements en SQL	Supprimer les enregistrements en MongoDB
<pre>DELETE FROM people WHERE status = "D"</pre>	<pre>db.people.deleteMany({ status: "D" })</pre>
<pre>DELETE FROM people</pre>	<pre>db.people.deleteMany({})</pre>

**PIPELINE D'AGREGATION MongoDB => OPERATION D'AGREGATION DE DONNEES
COURANTES EN SQL**

WHERE	\$match
GROUP BY	\$group
HAVING	\$match
SELECT	\$project
ORDER BY	\$sort
LIMIT	\$limit
SUM()	\$sum
COUNT()	\$sum \$sortByCount

⇒ *On travaille sur une collection nommée orders*

```
{
  cust_id: "abc123",
  ord_date: ISODate("2012-11-02T17:04:11.102Z"),
  status: 'A',
  price: 50,
  items: [
    { sku: "xxx", qty: 25, price: 1 },
    { sku: "yyy", qty: 25, price: 1 }
  ]
}
```

Instructions d'agrégation SQL	Instructions MongoDB
<pre>SELECT COUNT(*) AS count FROM orders</pre>	<pre>db.orders.aggregate([{ \$group: { _id: null,</pre>

```
count: { $sum: 1 }  
}  
},  
{  
  $project: {  
    _id: 0,  
    count: 1  
  }  
}  
])
```

Explication :

Pas de filtre sur `$match` à transmettre à `$group`.

Nous n'avons aucune contrainte pour `$group`

On va grouper via `$group` tous les documents dans la collection orders en un seul groupe de données, par `_id` mais c'est `null` donc pas de regroupement spécifique.

On utilise `$sum` pour calculer le total en ajoutant 1 à chaque document.

`$project`: affichage du résultat final sans `_id` car exclu.

On renomme le champ de comptage en "count".

Conclusion :

Le résultat de cette requête donnera le nombre total de documents dans la collection orders.

```
SELECT SUM(price) AS total  
FROM orders
```

```
db.orders.aggregate( [  
  {  
    $group: {  
      _id: null,  
      total: { $sum: "$price" }  
    }  
  }  
)  
{  
  $project: {  
    _id: 0,  
    total: 1  
  }  
}]
```

Explication :

Pas de filtre sur **\$match** à transmettre à **\$group**.

\$group : On regroupe tous les documents de la collection orders en un seul groupe.

_id: null pour indiquer qu'il n'y a pas de regroupement spécifique.

\$sum pour additionner les valeurs du champ "price" pour chaque document.

\$project: Affichage du résultat final. On renomme le champ de somme en "total".

Conclusion :

Le résultat de cette requête donnera la somme totale des prix dans la collection orders, avec le champ "total" comme résultat.

```
SELECT cust_id,  
       SUM(price) AS total  
FROM orders  
GROUP BY cust_id
```

```
db.orders.aggregate( [  
  {  
    $group: {  
      _id: "$cust_id",  
      total: { $sum: "$price" }  
    }  
  },  
  {  
    $project: {  
      cust_id: "$_id",  
      _id: 0,  
      total: 1  
    }  
  }  
])
```

Explication :

Pas de filtre sur **\$match** à transmettre à **\$group**.

Pour chaque **cust_id** on fait la somme sur les colonnes

\$group: on regroupe les documents de la collection **orders** en utilisant le champ "**cust_id**" comme clé de regroupement.

\$sum est utilisé pour calculer la somme des prix pour chaque groupe.

\$project: on projette le résultat final en renommant le champ "**_id**" en "**cust_id**" pour correspondre à la structure de la requête SQL.

Le champ "**total**" est conservé.

Conclusion :

Le résultat de cette requête donnera la somme des prix ("**total**") pour chaque client ("**cust_id**") dans la collection **orders**.

```
SELECT cust_id,  
       SUM(price) AS total  
FROM orders  
GROUP BY cust_id  
ORDER BY total
```

```
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$cust_id",  
      total: { $sum: "$price" }  
    }  
  },  
  {  
    $project: {  
      cust_id: "$_id",  
      _id: 0,  
      total: 1  
    }  
  },  
  {  
    $sort: { total: 1 }  
  }  
])
```

Explication :

\$group: On regroupe les documents de la collection orders en utilisant le champ "cust_id" comme clé de regroupement.

L'opérateur \$sum est utilisé pour calculer la somme des prix pour chaque groupe.

\$project: On projette le résultat final en renommant le champ "_id" en "cust_id" pour correspondre à la structure de la requête SGBDR SQL.

Le champ "total" est conservé.

\$sort: Cela trie les résultats en fonction de la valeur du champ "total".

La valeur 1 dans { total: 1 } signifie un tri croissant, tandis qu'une valeur -1 indiquerait un tri décroissant.

Conclusion :

Le résultat de cette requête donnera la somme des prix ("total") pour chaque client ("cust_id") dans la

```
SELECT cust_id,  
SUM(price) as total  
FROM orders  
WHERE status = 'A'  
GROUP BY cust_id
```

collection orders, triée par ordre croissant du total.

```
db.orders.aggregate([  
  {  
    $match: {  
      status: 'A'  
    }  
  },  
  {  
    $group: {  
      _id: '$cust_id',  
      total: { $sum: '$price' }  
    }  
  },  
  {  
    $project: {  
      cust_id: '$_id',  
      _id: 0,  
      total: 1  
    }  
  }  
])
```

Explication :

\$match: on filtre les documents dans la collection orders pour inclure uniquement ceux qui ont un statut égal à "A".

\$group: on regroupe les documents filtrés en utilisant le champ "cust_id" comme clé de regroupement.

\$sum est utilisé pour calculer la somme des prix pour chaque groupe.

\$project: on projette le résultat final en renommant le champ "_id" en "cust_id" pour correspondre à la structure de la requête SQL.

Le champ "total" est conservé.

Conclusion :

Le résultat de cette requête donnera la somme des prix ("total") pour chaque client ("cust_id") dans la collection orders pour les commandes ayant un statut "A".

Pour chaque cust_id unique avec le statut A, on additionne le champ de prix.

```
SELECT cust_id,  
       SUM(price) as total  
FROM orders  
WHERE status = 'A'  
GROUP BY cust_id  
HAVING total > 250
```

```
db.orders.aggregate([  
  {  
    $match: {  
      status: 'A'  
    }  
  },  
  {  
    $group: {  
      _id: '$cust_id',  
      total: { $sum: '$price' }  
    }  
  },  
  {  
    $match: {  
      total: { $gt: 250 }  
    }  
  },  
  {  
    $project: {  
      cust_id: '$_id',
```

```
_id: 0,  
total: 1  
}  
}  
])
```

Explication :

\$match: on filtre les documents dans la collection orders pour inclure uniquement ceux qui ont un statut égal à "A".

\$group: on regroupe les documents filtrés en utilisant le champ "cust_id" comme clé de regroupement.

\$sum est utilisé pour calculer la somme des prix pour chaque groupe.

\$match: on filtre les groupes en fonction de la valeur du champ "total", en ne conservant que ceux ayant un total supérieur à 250.

\$project: on projette le résultat final en renommant le champ "_id" en "cust_id" pour correspondre à la structure de la requête SQL.

Le champ "total" est conservé.

Conclusion :

Le résultat de cette requête donnera la somme des prix ("total") pour chaque client ("cust_id") dans la collection orders pour les commandes ayant un statut "A" et dont le total est supérieur à 250.